COMP.CS.510-2025-2026-1 Advanced Web Development: Back End

Group 201

Vilma Pirilä, vilma.pirila@tuni.fi

Elsi Häkkinen, elsi.hakkinen@tuni.fi

Juho Peltonen, juho.peltonen@tuni.fi

https://course-gitlab.tuni.fi/compcs510-spring2026/201/

Schedule

Week 1, March 9th to March 15th: Group formation, first meetup, checking project requirements, starting the documentation. Docker Compose setup.

Week 2, March 16th to 22nd: Second meetup. Picking the technologies, learning about NGINX, RabbitMQ and Docker. Randomizing the generator. Loose work division: Vilma in charge of frontend and communication to frontend + graphics, Elsi works in backend server setup and server communication including generator, Juho as the member with most experience handles docker files, does testing and works a full-stack role.

Week 3, March 23rd to March 29th: Work focus on server A and frontend. Group coding session during third meetup on the 26th.

Week 4, March 30th to April 5th: Easter holidays take over most of the week. Independent study. Graphics and server development. Due to incomplete testing, they were not pushed to the main branch yet.

Week 5, April 6th to April 12th: Checking that documentation is up to date. MIDWAY PROJECT CHECK on the 7th. Fourth meetup. Checking the branches + anything the members have worked on but not yet pushed to GitLab. Focus on server B.

Week 6, April 13th to April 19th: Finalizing server communication and adding graphics. Focus on frontend development.

Week 7, April 20th to April 28th: Final week of project, deadline on Tuesday 28th. Finalizing frontend and making other final fixes. Checking that documentation is up to date.

Members commit to spending a **minimum of 3 hours** on independent study and development each week, on top of **1-4 hours of weekly meetup**. Actual spent hours should aim to be above this, but fluctuation during the project is allowed, as long as the promised minimum is reached every week.

Architecture

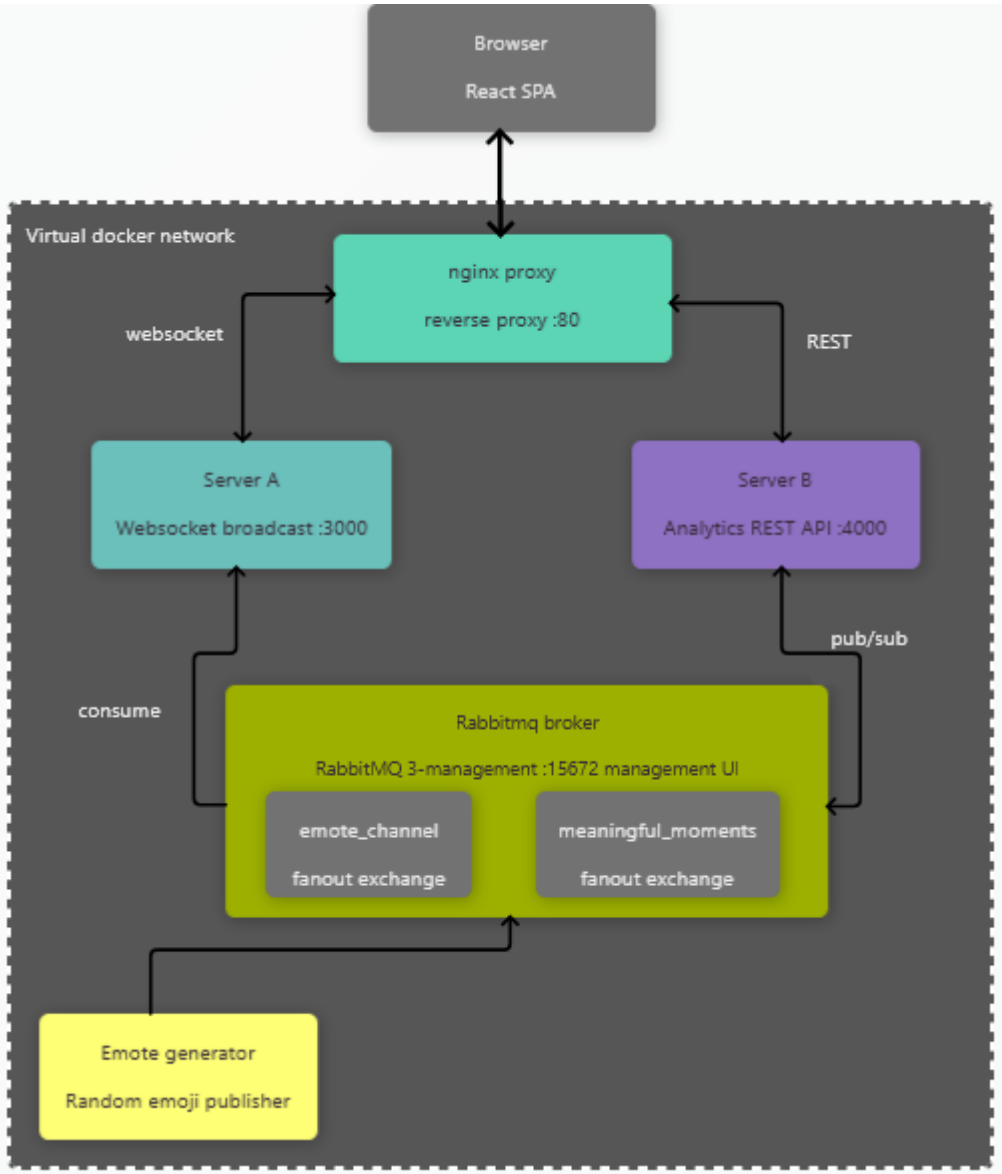

Diagram of the architeture

How to try the system:

- Ensure Docker is running on the device
- Open terminal in the project root folder
- Terminal: docker compose up –build
- Go to localhost in your browser

Server A and –B will fail to connect at first but automatically retry once rabbitmq initializes.

Frontend: React
Backend: Node.js
Considered but not used: Vue.js, TypeScript, vanilla JS.

These technologies were picked based on developers' familiarity with them, as we thought that this project had a lot of new things to learn without bringing in even more new tools. All of us knew React, while some of us had some familiarity with TypeScript, and only one of us had proper experience with Vue.js. We picked React over plain JavaScript for practical reasons, such as easier state management and UI updating.

Frontend: Single-Page Application built with React 18. It is served by an nginx web server running inside a Docker container. Its responsibilities are to deliver the compiled JavaScript application to the browser, and to act as a reverse proxy that routes browser WebSocket connections to Server A and REST API calls to Server.

Server A: Server A is a stateless event relay microservice. It is responsible for bridging asynchronous messages arriving from the RabbitMQ message broker to real-time browser clients over the WebSocket protocol. It only routes, without adding any business logic.

Server B: Server B is an event-driven analytics microservice, which processes real-time emoji reactions to detect meaningful moments based on bursts of reactions. RabbitMQ Broker handles its message routing. Server B connects to RabbitMQ, consumes emoji reactions, and detects spikes by maintaining a sliding time window.

Generator: The Generator produces a randomized continuous stream of emoji reaction events and publishes them to a RabbitMQ message exchange. Its purpose is to simulate real-time audience engagement traffic. It operates on a recursive async event loop.

Nginx

As recommended by the assignment, this project uses nginx as the single entry point to the entire application. It serves the React frontend and forwards requests to the right backend service based on the URL path.

Docker Compose

With Docker Compose, the entire project starts simply with the command 'docker compose up -- build' and tears down with 'docker compose down'. Each of the project's services has its own Dockerfile that defines how to build and run it.

Generator has a simple Node.js container that runs the emote publishing script, Server B has a Node.js container exposing port 4000 for the Express REST API, Server A has a Node.js container exposing port 3000 for the WebSocket server, and frontend has a two-stage build where it first builds the React app and then serves it with nginx.

RabbitMQ

We made the following decisions for RabbitMQ:

We decided that for a project where only one service is bound to the emote_channel, fanout should be fitting, as it is (if only a little) faster and there is less room for mistakes with its lower need for configuration. If the architecture had been more complex, we would have gone with topic for generator. We realize that if the project were to be scaled further, switch to topic would most likely be inevitable.

When it comes to acks, we used noACK: true for server B, and channel.ack(msg) for server A. As our service is a live emote stream, our reasoning is that emote data that is not current is not useful. Server B is in charge of determining when a meaningful moment happens, and if it crashes, the data it was handling is lost instead of forcing it to try and reprocess old information into possible meaningful moments (which risks polluting the sliding window). Server A only has the processed meaningful moments and it does not have to determine their internal time windows, so it makes sense for server A to be able to retrieve them even after a possible consumer crash. Thus, it uses manual ack.

RabbitMQ is an excellent fit for this project, as it enables the decoupling of architectural components – parts of architecture can be swapped out (such as emoji generator to a real live feed information) without necessarily needing to change other parts of architecture.

Design Notes

The goal was to keep the interface simple and easy to use.

The layout is divided into two main sections:

- Media Panel on the left showing the broadcasted video that is now just a small student-made animation. The emoji burst visualization is also shown on top of the video.

- Live Feed Panel on the right showing incoming meaningful events

The UI is simple, with two input fields for user input.

The user can change the time window (in milliseconds) and threshold by writing on the keyboard or toggling the increase and decrease (arrow) buttons with the mouse or with keyboard arrows. There is a save button that also activates when the Enter key is pressed. Feedback shows if the settings were saved or if there was an error.

The live feed has the emoji, its count, and time. In this case, the time shows hour, minute, and second, as it seemed right for this case. No need for milliseconds, day, month, etc.

The live feed panel is also scrollable so you can see the history without scrolling the whole page.

Icons are from Google Icons.

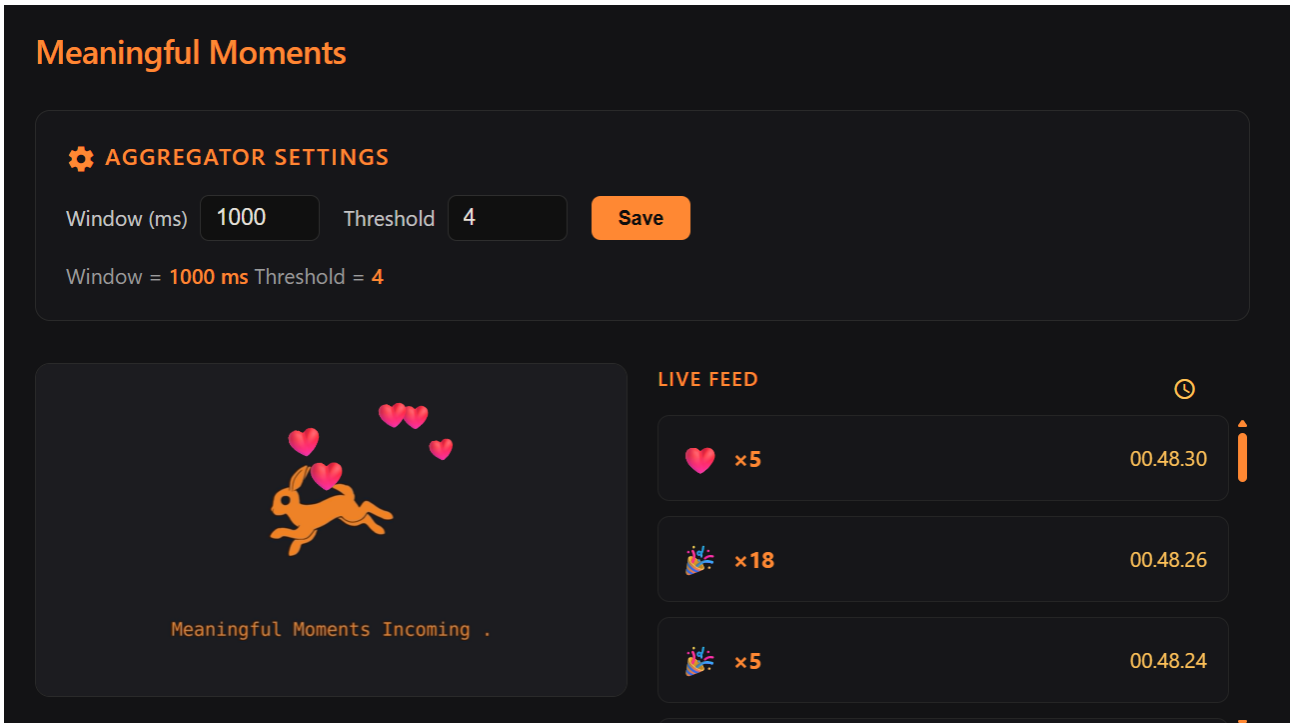

Figure 1 grafical user interface made with react

Learnings during project

Elsi:

Weeks 1-2: At the start of this project the learning curve for me was steep. I had only ever used Docker once before during the Software Engineer project course, and then it was mostly the master level students just telling me what to do, and I did not need to figure anything about it myself. I had never used NGINX or RabbitMQ. It took me a while to understand what they are and what their benefits are, such as RabbitMQ letting our services communicate without directly knowing about each other.

Weeks 3-4: Learning to use Docker and Docker Compose properly. It was during this part of the project that I started to routinely remember docker-related commands without needing to double-check before using them. I was also doing new things in backend; I had never used a time window before (windows_ms), but it seemed useful for determining our meaningful moments, so I tried implementing one. As I was unsure of my development decisions, I put my implementations in a branch so that they could be validated by other group members later.

Weeks 5-6: I learned more about APIs, retry logic and navigating Git. With our multi-part architecture, services needed to handle startup order gracefully, especially since RabbitMQ takes time to initialize; we became very aware of this issue when we were going through the project for midway check. This was a new problem for me, but both server A and server B were added logic to handle this.

Week 7: Git and GitLab functionality, such as dating issues for issue board and managing branches. During this part I made a double-check if we should change to topic or direct from the original fanout and learned more about each of their benefits. Preparing for the exam also made me more deeply aware of the theory behind our implementations and made me check our API setup more closely; our group decided that at this point the implementation was good enough for the scope and time constraints of this project, especially since the topic of possible changes was brought up with only 2-3 days before deadline. If I were to redo the APIs, I would add authentication keys and rethink the use of a HTTP server.

Juho:

Using RabbitMQ

Understanding how the AMQP model works in a real application. Fanout seemed like the simplest option to go with since routing keys are not required and technically its very efficient for raw message processing but does unnecessarily saturate the network and increase memory usage compared to routing. It would be trivial to add another server that analyzes the emotes as there would be no additional configuration on the generator side.

Nginx

I had not seen Nginx used before this project, but I see a lot of differences compared to more traditional servers (e.g. Apache). Its Event driven model is great for relatively high concurrencies where connections don't actually have something going on at all times. If a user sent an emote through Nginx it would be handled in the event loop and the loop would immediately afterwards handle the next user.

The server's SSL handling also seems interesting as SSL termination can be handled separately from the backend.

Mixing architectural models

REST is used for the control path for changing settings of the analytics server B as it's something that gets polled rarely.

Websockets are used in server A to consume meaningful moments and broadcast to all clients. This fits the use case.

Vilma:

Weeks 1-2: spend this time studying WebSockets and server-a. I spend some time watching videos about WebSockets, RabbitMQ and Docker. Docker was only one of which I had used earlier: mainly deleting images, stopping containers and docker compose up. I learned from Youtube and course material.

Weeks 3-4: Next I studied how nginx works. I never had heard about it either. We also changed vanilla frontend to react, because it was more familiar to me. I worked on the frontend docker file which was new for me and nginx configure file.

Weeks 5-6: Worked on the frontend and got to learn React again, but nothing actually new. Also wrote some design notes, trying to keep WCAG also in mind, but didn't. Tried to keep it simple and zoom-friendly, but didn't want to use too much time on frontend because it's not this course's topic.

Week 7: Checked that GitLab issues were on schedule and documentation. Last frontend tweaks. I went through the whole code to understand it.